The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch08_unsupervised

AYuSh

Contents

Chapter 8: Unsupervised Learning

Unsupervised learning is where ML meets data with no answer key: only x, no y, and the job is to find structure — groups, densities, low-dimensional organization, exceptions. Interviewers use it for two things. First, the algorithms themselves: k-means and PCA are among the most-asked derivations in ML interviews, EM is a rite of passage, and "how would you choose the number of clusters?" is a screening staple. Second — and this is what separates candidates — the *epistemics*: without labels there is no accuracy to compute, so every claim ("these are the customer segments") requires justifying an evaluation strategy, and "how would you know your clustering is good?" has ended more interviews than any derivation. Chapter 4 flagged this slipperiness; here it gets teeth.

The chapter covers clustering in four flavors (k-means's centroid geometry, hierarchical's merge tree, DBSCAN's density reachability, GMM's probabilistic mixtures — each with a different implicit definition of "cluster", which is the real lesson), the EM algorithm underneath mixtures, dimensionality reduction from linear (PCA, LDA) to neighbor-preserving (t-SNE, UMAP), association rule mining with its support/confidence/lift triple, and anomaly detection. Everything is implemented and measured in the listings — including the honest failures.

K-Means

K-means partitions n points into k clusters by choosing centroids $\mu_1, \ldots, \mu_k$ minimizing **inertia** — within-cluster sum of squared distances:

$$J = \sum_{i=1}^{n} \min_{j} \|x_i - \mu_j\|^2$$

Exact minimization is NP-hard; **Lloyd's algorithm** is the universal heuristic: (1) assign each point to its nearest centroid; (2) move each centroid to the mean of its assigned points; repeat until nothing moves. Both steps provably decrease J — the assignment step is optimal given centroids, the update step is optimal given assignments (the mean minimizes summed squared distance, a Chapter 1 fact) — so the algorithm always converges. But only to a *local* minimum, and which one depends entirely on initialization: Listing 1's twenty random-restart runs land anywhere in a 1.24× inertia spread. The standard fixes are restarts (keep the best of n_init runs) and **k-means++ initialization** — seed the first centroid uniformly, then pick each next centroid with probability proportional to its squared distance from the nearest existing seed, spreading the seeds across the data with a provable O(log k) approximation guarantee. Listing 1's k-means++ run matches the best of twenty random restarts on the first try.

Choosing k — the inevitable follow-up — has no oracle, only heuristics. The **elbow method** plots inertia against k and looks for the bend: inertia decreases monotonically in k (more centroids can only shorten distances — at k = n it is zero, uselessly), so what carries information is the *rate* slowing. Listing 2 shows drops of ~44% per increment up to the true k = 6, collapsing to ~6% after — an unusually clean elbow; real elbows are mushier, which is the criticism to volunteer. The **silhouette score** is sturdier: for each point, with a = mean distance to its own cluster and b = mean distance to the nearest other cluster, $s = (b - a)/\max(a, b) \in [-1, 1]$; average over points. It rewards tight, well-separated clusters and *can decrease* — Listing 2's silhouette peaks exactly at k = 6 (0.662). Honest caveats: silhouette presumes compact-ish geometry (it will prefer 2 super-clusters when groups overlap — a scale-of-analysis ambiguity, not a bug), costs $O(n^2)$ distances, and neither method beats domain knowledge about how many segments the business can act on. Gap statistic and BIC-via-GMM are the other names worth having.

Limitations are the richest k-means interview territory, because every one traces back to the objective: squared Euclidean distance to a single centroid presumes clusters are **spherical, similar in size and density, and convex**. Listing 3 breaks each assumption and measures the damage (adjusted Rand index against truth): two moons — k-means 0.249, because no centroid pair can represent interlocking crescents (DBSCAN: 1.000); sheared anisotropic Gaussians — 0.430, because "nearest centroid" draws boundaries perpendicular to the wrong axes (GMM with full covariance: 1.000); one big diffuse cluster plus two small tight ones — 0.252, because splitting the big cluster lowers inertia more than respecting the small ones (GMM: 0.985). Further standing issues: sensitivity to feature scale (it's a distance method — standardize, Chapter 6's rule), sensitivity to outliers (means get dragged; k-medoids uses actual points as centers), hard assignments (no "60% cluster A" — GMM's opening), and the requirement to pick k up front. Say the summary sentence: *k-means doesn't find your clusters; it finds the best spherical-Voronoi story it can tell, whether or not your data is one.*

Two closers interviewers like: k-means is exactly a GMM with equal spherical covariances and hard assignments (the EM connection below makes this precise), and at scale it runs as mini-batch k-means (stochastic centroid updates — the SGD of clustering) and powers vector-quantization workhorses from codebooks to product-quantized vector indexes (Chapter 24).

Hierarchical clustering

Hierarchical clustering returns not one partition but a **dendrogram** — a binary merge tree over the data, cuttable at any height into any number of clusters. **Agglomerative** (bottom-up: start with n singletons, repeatedly merge the two closest clusters) is the standard; **divisive** (top-down recursive splitting) is rarer and mostly conceptual. No k is needed to build the tree; k is chosen afterward by cutting — and a large jump in merge

distance marks a natural cut (Listing 4: merge distances 13.0 → 85.1 → 95.3; the 13→85 jump says "three clusters", and cutting there recovers the true sizes exactly).

Everything hinges on the **linkage** — the definition of distance between clusters: **single** (closest pair — follows chains and filaments, can "leak" through noise bridges), **complete** (farthest pair — compact, similar-diameter clusters, outlier-sensitive), **average** (mean over all pairs — the compromise, UPGMA), and **ward** (merge the pair whose union least increases within-cluster variance — inertia-greedy, k-means's sibling, the usual default). The choice is a prior about cluster shape, and Listing 4 stages the clean double dissociation: on compact blobs, ward/average/complete score ARI 0.985–0.993 while single scores 0.571 (chaining through inter-cluster noise); on two moons, single scores a perfect 1.000 (chaining along the crescents is exactly right) while ward scores 0.162 (compactness is exactly wrong). Neither is "better" — they define "cluster" differently, and saying that is the answer.

Practicalities: $O(n^2)$ memory for the distance matrix and $O(n^2 \log n)$-ish time bound it to tens of thousands of points; it inherits distance's scaling requirements; it is deterministic (no init lottery); and the dendrogram itself is often the deliverable — taxonomies, gene-expression heatmap orderings, topic hierarchies — a reason to choose it even when a flat partition would score the same.

DBSCAN and HDBSCAN

DBSCAN defines clusters by **density, not distance to a center**: a **core point** has ≥ min_samples neighbors within radius ε; points within ε of a core point are **density-reachable**; clusters are maximal chains of density-reachable points; everything else is **noise** — a first-class output, not a failure. The consequences interviewers probe: no k required (the data's density structure decides), arbitrary cluster shapes (moons, rings, filaments — anything density-connected; Listing 5: ARI 0.970 on noisy moons where k-means got 0.249), and native outlier flagging (the injected 10% uniform noise lands in the −1 label).

The cost is ε, and it is a cliff, not a dial: Listing 5's sweep shows ε = 0.05 shattering the data into 34 fragments with 275 "noise" points (ARI ≈ 0), ε = 0.1–0.2 nearly perfect, ε = 0.4 fusing everything into one blob (ARI 0.123). The k-distance plot (sorted distance to each point's k-th neighbor; look for the knee) is the standard ε-picking heuristic. The deeper failure mode: **one global ε cannot serve clusters of different densities** — tight enough for the dense cluster shatters the sparse one; loose enough for the sparse one fuses the dense pair. That is precisely what **HDBSCAN** fixes: it builds a hierarchy over *all* density levels (conceptually, DBSCAN at every ε at once), then extracts the most stable clusters across the hierarchy — no ε at all, just min_cluster_size. Listing 5: HDBSCAN matches tuned DBSCAN (ARI 0.967) with nothing to tune, which is why it has become the practical default for density clustering. Remaining shared caveats: distance-

based (scale features; degrades in high dimension like KNN — Chapter 6), and border-point assignments can vary between runs in some implementations.

Gaussian Mixture Models and EM

A GMM says the data was generated by a two-stage story: pick component j with probability π_j, then draw $x \sim \mathcal{N}(\mu_j, \Sigma_j)$. The density is a weighted sum of Gaussians:

$$p(x) = \sum_{j=1}^{k} \pi_j \, \mathcal{N}(x \mid \mu_j, \Sigma_j)$$

Clustering becomes posterior inference — the **responsibility** $r_{ij} = p(\text{component } j \mid x_i)$ by Bayes' rule — and assignments are **soft**: a point between two components is 60/40, not forced to choose. Full covariance matrices let components be ellipses at any orientation, which is exactly what rescued Listing 3's anisotropic case (ARI 1.000 vs k-means 0.430). Covariance structure is the capacity dial: spherical ⊂ diag ⊂ tied ⊂ full, trading flexibility against parameters-per-component (full costs $O(d^2)$ each — in high d, regularize or restrict). And because a GMM is a *generative density model*, you can sample new points, score likelihoods (→ anomaly detection below), and select k with honest information criteria: BIC/AIC penalize the likelihood by parameter count, something inertia can never do.

Direct maximum likelihood is intractable — the log of a *sum* doesn't decompose — because the component labels are **latent variables**. **EM (Expectation-Maximization)** is the standard escape, and it alternates two readable steps. **E-step**: with parameters fixed, compute responsibilities r_{ij} (Bayes' rule — "which component probably made this point?"). **M-step**: with responsibilities fixed, update parameters by responsibility-weighted MLE — π_j = average responsibility, μ_j = weighted mean, Σ_j = weighted covariance; each formula is Chapter 1's Gaussian MLE with fractional counts. The guarantee that makes EM famous: **each iteration cannot decrease the observed-data log-likelihood** (it maximizes a tight lower bound — Jensen's inequality — that touches the likelihood at the current parameters). Listing 6 implements the full loop from scratch in ~25 lines with the monotonicity asserted programmatically: log-likelihood climbs $-7746 \to -2213 \to -2161$, converges in 21 steps, and recovers the true mixture (π, μ, σ all within noise, matching sklearn). The k-means connection completes the circuit: freeze all covariances to εI, let $\varepsilon \to 0$, and responsibilities harden to nearest-centroid assignments — EM degenerates into exactly Lloyd's algorithm. K-means is a GMM that stopped believing in uncertainty.

EM's caveats are interview follow-ups: local maxima (initialize from k-means, use restarts); **singularities** — a component can collapse onto one point, $\sigma \to 0$, likelihood $\to \infty$, which is why implementations floor the covariance (reg_covar); label switching (component identities are arbitrary — sort before comparing, as Listing 6 does); and convergence can be slow when components overlap. EM itself is general far beyond

GMMs — missing data, hidden Markov models, any latent-variable likelihood — and "derive EM for a mixture" is a canonical hard-round request that Listing 6 is designed to rehearse.

Dimensionality reduction: PCA, LDA, t-SNE, UMAP

PCA finds the orthogonal directions of maximal variance and projects onto the top k. Equivalent formulations worth being able to swap between: (1) maximize projected variance; (2) minimize squared reconstruction error (the best rank-k linear approximation — the two objectives are the same by Pythagoras); (3) eigendecompose the covariance matrix — or, numerically better, **SVD the centered data**: $X_c = USV^\top$, principal axes = rows of $V^\top$, component variances = $S^2/(n-1)$. Centering is mandatory (uncentered "PCA" finds the direction to the mean, not of the spread); scaling matters exactly as in every distance method — standardize unless features share units, or the largest-variance raw feature *is* the first PC by default. **Explained variance ratio** guides k: Listing 7 on 64-pixel digits — 10 components keep 73.8%, 40 keep 98.8% — and the from-scratch SVD matches sklearn exactly, with reconstruction demonstrated (project to 10-D, map back, pixel MSE 4.91). Uses beyond visualization: decorrelation/whitening, noise reduction, compression before distance methods (KNN, k-means — Chapter 6's curse mitigation), and multicollinearity removal for regression (Chapter 5). Limits: linear (a Swiss-roll manifold defeats it), variance ≠ importance (the discriminative direction may be PC 40 — see LDA), and components are combinations of everything, costing interpretability.

LDA (Linear Discriminant Analysis) is the supervised counterpoint: it uses *labels* to find ≤ (classes − 1) directions maximizing between-class variance over within-class variance — separation, not spread. Listing 8 makes the contrast quantitative on digits: 2-D PCA supports 15-NN accuracy 0.641; 2-D LDA, 0.687 — better, because its two axes were chosen to separate the ten classes, not to preserve pixel variance. LDA belongs in this chapter as the reminder that "dimensionality reduction" is not inherently unsupervised — when labels exist and the goal is class structure, use them.

t-SNE and UMAP abandon linearity and global fidelity to preserve *neighborhoods*. t-SNE converts pairwise distances into neighbor probabilities (bandwidths set per point by **perplexity**, effectively "how many neighbors count" — the main knob, typically 5–50), then arranges points in 2-D so the low-dimensional neighbor distribution (heavy-tailed Student-t, which is what prevents the crowding problem of squeezing high-D neighborhoods into 2-D) matches via KL-divergence minimization. UMAP builds a fuzzy k-NN graph and optimizes a cross-entropy layout — similar spirit, better preserved global structure, faster on large n, and it can transform *new* points (t-SNE classically cannot — a production-relevant difference). The payoff is dramatic: Listing 8's digits go from 0.64 (PCA) to 0.976/0.978 (t-SNE/UMAP) in 2-D 15-NN accuracy — local class structure almost perfectly preserved. The mandatory warnings, straight from interview scripts: **cluster sizes and inter-cluster distances in a t-SNE plot are not meaningful** (the algorithm equalizes densities; distances between islands are artifacts), different

perplexities tell different stories (always show more than one), it will happily draw "clusters" in pure noise, and axes have no interpretation. These are *visualization and neighbor-structure* tools — cluster on them with care (HDBSCAN-on-UMAP is a popular, effective, and theoretically awkward pipeline; know both facts), and never feed t-SNE coordinates into downstream models as features.

Association rule mining

Market-basket analysis mines co-occurrence rules A → B from transaction sets. The three metrics, on baskets containing itemsets:

- **Support(A → B)** = P(A ∪ B in basket) — how common the pattern is; filters rare noise.
- **Confidence(A → B)** = P(B | A) = support(A∪B)/support(A) — how reliable the implication is.
- **Lift(A → B)** = confidence / support(B) = P(A∪B)/(P(A)P(B)) — how much more than *chance* the pair co-occurs. Lift > 1: complements; = 1: independent; < 1: substitutes. Lift is the honesty check: Listing 9's beer → milk rule has confidence 0.80 (looks strong!) and lift 1.00 — milk is just in 80% of baskets; the rule says nothing. Meanwhile beer → diapers, confidence 0.80, lift 1.33 — a real association. Confidence-without-lift is the classic rookie reading, and catching it is the entire point of the question.

Apriori finds all frequent itemsets level-wise using the **downward-closure property**: every subset of a frequent itemset is frequent, so k-itemsets need only be generated from frequent (k−1)-itemsets, pruning the exponential lattice (Listing 9 implements the first two levels from scratch: 5 frequent items → 5 frequent pairs at 30% support, then scores all rules). **FP-Growth** reaches the same answer without candidate generation: compress transactions into a prefix tree (FP-tree) sharing common prefixes, then mine it recursively — typically much faster on large sparse baskets. Modern framing to volunteer: association rules survive in retail analytics and rule-based recommendation baselines, and lift's logic (observed co-occurrence over independence expectation) is the same idea as PMI in NLP (Chapter 19).

Anomaly detection

Anomaly detection asks "which points don't belong?" — usually unsupervised, because anomalies are rare, unlabeled, and unlike each other (fraud, sensor faults, intrusions). The three classical detectors triangulate different definitions of "doesn't belong", and Listing 10 scores them on blobs + scattered outliers:

- **Isolation Forest** (0.82/0.82 precision/recall): anomalies are *easy to isolate* — random axis-aligned splits strand an outlier in few cuts, so short average path

length across random trees = anomalous. O(n log n), no distances (scale-tolerant, high-d-tolerant), the industrial default.
- **One-Class SVM** (0.71/0.70): learns a boundary around the "normal" region in kernel space; ν bounds the outlier fraction. Elegant, but kernel-scaling and γ-sensitivity make it the fussiest of the three.
- **LOF (Local Outlier Factor)** (0.84/0.84): compares each point's local density to its neighbors' — a point in a sparse spot *relative to its neighborhood* is anomalous. Catches local anomalies global methods miss (a point at a dense cluster's edge); inherits KNN's scaling/dimension issues; classically has no natural predict-on-new-data mode (novelty=True changes the contract).

Two more families complete the answer: **GMM/density scoring** — fit the density, flag low-likelihood points (natural when you already have a generative model), and **autoencoders** — train a bottlenecked reconstructor on (mostly) normal data; anomalies reconstruct poorly, and reconstruction error is the score (Chapter 18 builds them; the idea works because the bottleneck learns the normal manifold and has no capacity budget for rarities). The shared operational truths: **contamination is a chosen alarm rate**, not a discovered fact — you are picking a point on the precision/recall curve (Listing 10's knob), and thresholds should be set against the cost of missed-vs-false alarms with whatever labeled examples exist (Chapter 10's machinery); evaluation without labels falls back on injection studies (plant synthetic anomalies, measure recovery — exactly Listing 10's design) and human triage of the top-k alarms.

Code implementations

Every listing was executed as shown; outputs are real. The set implements the chapter: Lloyd's algorithm and the init lottery, elbow vs silhouette on known k, three staged k-means failures with the model that fixes each, the linkage double dissociation, DBSCAN's ε cliff and HDBSCAN's escape from it, EM from scratch with monotonicity asserted, PCA via SVD with reconstruction, the four-way embedding bake-off, Apriori with the lift lesson, and three anomaly detectors scored against injected truth.

Listing 1 — K-means from scratch and the initialization lottery

Lloyd's two-step alternation in twelve lines. Twenty random initializations spread across a 1.24× inertia range — same data, same algorithm, different local minima. k-means++ hits the best observed value in one seeded try.

```
"""Listing 1: k-means from scratch (Lloyd's algorithm), and why initialization
matters."""
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=1500, centers=6, cluster_std=1.2, random_state=0)

def kmeans(X, k, init_centers, iters=100):
```

```
    C = init_centers.copy()
    for _ in range(iters):
        # Assignment step: each point to its nearest center
        d = ((X[:, None, :] - C[None, :, :]) ** 2).sum(-1)     # (n, k) sq-distances
        labels = d.argmin(1)
        # Update step: each center to the mean of its points
        newC = np.array([X[labels == j].mean(0) if (labels == j).any() else C[j]
                         for j in range(k)])
        if np.allclose(newC, C): break
        C = newC
    inertia = ((X - C[labels]) ** 2).sum()
    return labels, C, inertia

rng = np.random.default_rng(1)
# Random init, 20 restarts: the spread shows the local-minimum lottery
inertias = []
for _ in range(20):
    init = X[rng.choice(len(X), 6, replace=False)]
    *_, inertia = kmeans(X, 6, init)
    inertias.append(inertia)
print(f"random init, 20 runs: best {min(inertias):.0f}  worst {max(inertias):.0f}  "
      f"({max(inertias)/min(inertias):.2f}x spread)")

sk = KMeans(n_clusters=6, init="k-means++", n_init=10, random_state=1).fit(X)
print(f"k-means++ (sklearn) : inertia {sk.inertia_:.0f}")
```

Output:

```
random init, 20 runs: best 3518  worst 4359  (1.24x spread)
k-means++ (sklearn) : inertia 3518
```

Listing 2 — Choosing k: elbow vs silhouette on known k=6

Inertia falls forever; its *percentage drop* collapses from ~44% to ~6% right after the true k — the elbow. Silhouette, which can decrease, peaks exactly at k=6. On real data the elbow is rarely this crisp; the silhouette peak is the sturdier signal.

```
"""Listing 2: choosing k -- elbow (inertia) vs silhouette, on data with known k=6."""
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X, _ = make_blobs(n_samples=1500, centers=6, cluster_std=1.0,
                  center_box=(-15, 15), random_state=0)

print(f"{'k':>3} {'inertia':>9} {'drop %':>7} {'silhouette':>11}")
prev = None
for k in range(2, 11):
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    sil = silhouette_score(X, km.labels_)
    drop = f"{100*(prev-km.inertia_)/prev:>6.1f}%" if prev else "      -"
    print(f"{k:>3} {km.inertia_:>9.0f} {drop} {sil:>11.3f}")
    prev = km.inertia_
```

Output:

```
  k   inertia  drop %  silhouette
  2     27760       -       0.589
```

```
  3     15334   44.8%       0.552
  4      8861   42.2%       0.566
  5      4953   44.1%       0.621
  6      2785   43.8%       0.662
  7      2615    6.1%       0.591
  8      2438    6.8%       0.549
  9      2291    6.0%       0.472
 10      2130    7.0%       0.420
```

Listing 3 — Three staged k-means failures, each with its cure

Adjusted Rand index against known truth. Moons: no two centroids can represent interlocking crescents — DBSCAN's density chains can. Anisotropic: nearest-centroid boundaries cut across the stretched clusters — full-covariance GMM tilts its ellipses to match. Unequal sizes: splitting the big diffuse cluster buys more inertia than respecting the small tight ones — GMM's per-component variances fix it.

```
"""Listing 3: k-means limitations -- when the spherical assumption is the wrong
prior."""
import numpy as np
from sklearn.datasets import make_moons, make_blobs
from sklearn.cluster import KMeans, DBSCAN
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score as ari

rng = np.random.default_rng(2)

# Case 1: non-convex shapes (two moons)
Xm, ym = make_moons(n_samples=600, noise=0.06, random_state=2)
km = KMeans(2, n_init=10, random_state=2).fit_predict(Xm)
db = DBSCAN(eps=0.2, min_samples=5).fit_predict(Xm)
print("two moons     : k-means ARI", f"{ari(ym, km):.3f}",
      "  DBSCAN ARI", f"{ari(ym, db):.3f}")

# Case 2: anisotropic (stretched) Gaussians
Xb, yb = make_blobs(n_samples=600, centers=3, random_state=3)
Xa = Xb @ np.array([[0.6, -0.64], [-0.4, 0.85]])          # shear the space
km = KMeans(3, n_init=10, random_state=3).fit_predict(Xa)
gm = GaussianMixture(3, covariance_type="full", n_init=10,
random_state=3).fit_predict(Xa)
print("anisotropic   : k-means ARI", f"{ari(yb, km):.3f}",
      "  GMM(full) ARI", f"{ari(yb, gm):.3f}")

# Case 3: unequal cluster sizes/densities
Xu, yu = make_blobs(n_samples=[500, 60, 40], centers=[[0,0],[5,5],[9,0]],
                    cluster_std=[2.2, 0.5, 0.5], random_state=4)
km = KMeans(3, n_init=10, random_state=4).fit_predict(Xu)
gm = GaussianMixture(3, covariance_type="full", n_init=10,
random_state=4).fit_predict(Xu)
print("unequal sizes : k-means ARI", f"{ari(yu, km):.3f}",
      "  GMM(full) ARI", f"{ari(yu, gm):.3f}")
```

Output:

```
two moons     : k-means ARI 0.249   DBSCAN ARI 1.000
anisotropic   : k-means ARI 0.430   GMM(full) ARI 1.000
unequal sizes : k-means ARI 0.252   GMM(full) ARI 0.985
```

Listing 4 — Linkage choices: a clean double dissociation

On compact blobs, ward/average/complete are near-perfect and single linkage chains through the gaps (0.571). On moons, single linkage is *perfect* (chaining along a crescent is exactly right) and ward is worst (0.162 — compactness is exactly wrong). The linkage is a prior about cluster shape. The dendrogram's merge distances then locate the natural cut: 13.0 → 85.1 is the jump, so cut into three.

```
"""Listing 4: agglomerative clustering -- linkage choices are different definitions of
'close'."""
import numpy as np
from scipy.cluster.hierarchy import linkage, fcluster
from sklearn.datasets import make_blobs, make_moons
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import adjusted_rand_score as ari

# Compact blobs: everything works; ward is the default for a reason
Xb, yb = make_blobs(n_samples=400, centers=[[0, 0], [7, 4], [1, 8]], cluster_std=1.0,
                    random_state=5)
# Elongated chains (moons): single linkage's chaining is a feature here, ward's
compactness a bug
Xm, ym = make_moons(n_samples=400, noise=0.05, random_state=5)

print(f"{'linkage':>9} {'blobs ARI':>10} {'moons ARI':>10}")
for link in ["single", "complete", "average", "ward"]:
    ab = AgglomerativeClustering(3, linkage=link).fit_predict(Xb)
    am = AgglomerativeClustering(2, linkage=link).fit_predict(Xm)
    print(f"{link:>9} {ari(yb, ab):>10.3f} {ari(ym, am):>10.3f}")

# The dendrogram is the real output: cut it anywhere for any number of clusters
Z = linkage(Xb, method="ward")          # (n-1) merges: [idx_a, idx_b, distance, size]
print("\nlast 3 merges (ward), distances:", np.round(Z[-3:, 2], 1))
for k in [2, 3, 4]:
    labels = fcluster(Z, t=k, criterion="maxclust")
    print(f"cut at k={k}: cluster sizes {np.bincount(labels)[1:]}")
```

Output:

```
  linkage  blobs ARI  moons ARI
   single      0.571      1.000
 complete      0.985      0.389
  average      0.993      0.259
     ward      0.993      0.162

last 3 merges (ward), distances: [13.  85.1 95.3]
cut at k=2: cluster sizes [134 266]
cut at k=3: cluster sizes [134 132 134]
cut at k=4: cluster sizes [134 132  39  95]
```

Listing 5 — DBSCAN's ε cliff, and HDBSCAN's escape

Moons plus 10% uniform noise. ε=0.05 shatters (34 fragments, ARI ≈ 0); ε=0.1–0.2 nails it and flags the noise; ε=0.4 fuses everything. HDBSCAN gets the same quality with no ε — min_cluster_size is its only real knob.

```
"""Listing 5: DBSCAN -- density clustering with noise, its eps cliff, and HDBSCAN's
fix."""
```

```
import numpy as np
from sklearn.cluster import DBSCAN, HDBSCAN
from sklearn.datasets import make_moons
from sklearn.metrics import adjusted_rand_score as ari

rng = np.random.default_rng(6)
Xm, ym = make_moons(n_samples=600, noise=0.06, random_state=6)
X = np.vstack([Xm, rng.uniform(-1.5, 2.5, (60, 2))])          # add 10% uniform noise
y = np.hstack([ym, np.full(60, -1)])                          # noise gets label -1

print(f"{'eps':>5} {'clusters':>9} {'noise pts':>10} {'ARI':>6}")
for eps in [0.05, 0.1, 0.2, 0.4]:
    lab = DBSCAN(eps=eps, min_samples=5).fit_predict(X)
    n_clusters = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"{eps:>5} {n_clusters:>9} {(lab == -1).sum():>10} {ari(y, lab):>6.3f}")

hdb = HDBSCAN(min_cluster_size=15).fit_predict(X)
n_clusters = len(set(hdb)) - (1 if -1 in hdb else 0)
print(f"\nHDBSCAN: {n_clusters} clusters, {(hdb == -1).sum()} noise pts, "
      f"ARI {ari(y, hdb):.3f}  -- no eps to tune")
```

Output:

```
  eps  clusters  noise pts    ARI
 0.05        34        275 -0.016
  0.1         2         59  0.970
  0.2         2         44  0.948
  0.4         1         32  0.123

HDBSCAN: 2 clusters, 52 noise pts, ARI 0.967  -- no eps to tune
```

Listing 6 — EM for a Gaussian mixture, from scratch

The complete E-step (responsibilities via Bayes) and M-step (responsibility-weighted MLE), with EM's defining guarantee asserted in code: the log-likelihood never decreases. Converges in 21 steps and recovers the true mixture, matching sklearn.

```
"""Listing 6: EM for a two-component 1-D Gaussian mixture, from scratch."""
import numpy as np
from sklearn.mixture import GaussianMixture

rng = np.random.default_rng(7)
# True mixture: 40% N(-2, 0.8^2) + 60% N(3, 1.5^2)
x = np.hstack([rng.normal(-2, 0.8, 400), rng.normal(3, 1.5, 600)])

def norm_pdf(x, mu, sd):
    return np.exp(-0.5 * ((x - mu) / sd) ** 2) / (sd * np.sqrt(2 * np.pi))

# init: crude split at the extremes
pi, mu, sd = np.array([0.5, 0.5]), np.array([x.min(), x.max()]), np.array([1.0, 1.0])
prev_ll = -np.inf
for step in range(200):
    # E-step: responsibilities -- P(component | point), Bayes' rule
    dens = np.stack([pi[k] * norm_pdf(x, mu[k], sd[k]) for k in range(2)])  # (2, n)
    r = dens / dens.sum(0)
    # M-step: weighted MLE updates
    Nk = r.sum(1)
    pi = Nk / len(x)
    mu = (r * x).sum(1) / Nk
    sd = np.sqrt((r * (x - mu[:, None]) ** 2).sum(1) / Nk)
    ll = np.log(dens.sum(0)).sum()
```

```
    if step in (0, 1, 4, 20):
        print(f"step {step:>3}: log-likelihood {ll:>10.2f}")
    assert ll >= prev_ll - 1e-9, "EM must never decrease the log-likelihood"
    if ll - prev_ll < 1e-8: break
    prev_ll = ll

print(f"converged at step {step}: ll {ll:.2f}")
print(f"scratch : pi={np.round(pi,3)} mu={np.round(mu,2)} sd={np.round(sd,2)}")
sk = GaussianMixture(2, random_state=7).fit(x[:, None])
order = np.argsort(sk.means_.ravel())
print(f"sklearn : pi={np.round(sk.weights_[order],3)} mu={np.round(sk.means_.ravel()
[order],2)} "
      f"sd={np.round(np.sqrt(sk.covariances_.ravel()[order]),2)}")
print(f"truth   : pi=[0.4 0.6] mu=[-2.  3.] sd=[0.8 1.5]")
```

Output:

```
step   0: log-likelihood   -7746.58
step   1: log-likelihood   -2213.23
step   4: log-likelihood   -2163.52
step  20: log-likelihood   -2160.60
converged at step 21: ll -2160.60
scratch : pi=[0.399 0.601] mu=[-2.08  2.92] sd=[0.74 1.44]
sklearn : pi=[0.401 0.599] mu=[-2.08  2.93] sd=[0.75 1.43]
truth   : pi=[0.4 0.6] mu=[-2.  3.] sd=[0.8 1.5]
```

Listing 7 — PCA from scratch via SVD

Center, SVD, read off axes and variances — and the explained-variance ratios match sklearn boolean-exactly. The reconstruction round trip (project to 10-D, map back to 64-D) is the "PCA as best rank-k approximation" claim, measured.

```
"""Listing 7: PCA from scratch via SVD -- variance explained, reconstruction, sklearn
parity."""
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

X, _ = load_digits(return_X_y=True)          # 1797 digits, 64 pixels
Xc = X - X.mean(0)                           # center -- mandatory for PCA

# SVD of the centered data: principal axes = rows of Vt, scores = U*S
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
var = S**2 / (len(X) - 1)                    # eigenvalues of the covariance matrix
evr = var / var.sum()                        # explained variance ratio
print("explained variance ratio, first 5 PCs:", np.round(evr[:5], 3))
for k in [2, 10, 20, 40]:
    print(f"  top {k:>2} PCs keep {evr[:k].sum():.1%} of variance")

# Reconstruction: project to k dims, map back, measure pixel error
k = 10
Z = Xc @ Vt[:k].T                            # (n, k) compressed representation
X_rec = Z @ Vt[:k] + X.mean(0)               # back to 64-D
mse = np.mean((X - X_rec) ** 2)
print(f"\nreconstruction MSE with k=10: {mse:.2f} (pixel values 0-16)")

sk = PCA(n_components=5).fit(X)
print("sklearn EVR matches:", np.allclose(sk.explained_variance_ratio_, evr[:5]))
```

Output:

```
explained variance ratio, first 5 PCs: [0.149 0.136 0.118 0.084 0.058]
  top  2 PCs keep 28.5% of variance
  top 10 PCs keep 73.8% of variance
  top 20 PCs keep 89.4% of variance
  top 40 PCs keep 98.8% of variance

reconstruction MSE with k=10: 4.91 (pixel values 0-16)
sklearn EVR matches: True
```

Listing 8 — Four ways to 2-D: PCA vs LDA vs t-SNE vs UMAP

The quality metric is 15-NN accuracy *in the 2-D embedding* — how much class-relevant neighborhood structure survived the trip down from 64-D. Linear methods keep ~0.64–0.69; the neighbor-preserving nonlinear methods keep ~0.98. The price: t-SNE/UMAP coordinates are for looking at, not for measuring distances or feeding models.

```
"""Listing 8: PCA vs LDA vs t-SNE vs UMAP -- what survives the trip to 2-D."""
import warnings, time
warnings.filterwarnings("ignore")
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.manifold import TSNE
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
import umap

X, y = load_digits(return_X_y=True)

def knn_quality(Z, y):
    """How well do 2-D neighborhoods preserve class structure? 15-NN accuracy."""
    Z_tr, Z_te, y_tr, y_te = train_test_split(Z, y, test_size=0.3, random_state=8)
    return KNeighborsClassifier(15).fit(Z_tr, y_tr).score(Z_te, y_te)

methods = {
    "PCA (2)": lambda: PCA(2).fit_transform(X),
    "LDA (2)": lambda: LDA(n_components=2).fit_transform(X, y),   # supervised!
    "t-SNE": lambda: TSNE(2, perplexity=30, random_state=8).fit_transform(X),
    "UMAP": lambda: umap.UMAP(n_components=2, random_state=8).fit_transform(X),
}
print(f"{'method':<8} {'seconds':>8} {'2-D 15-NN acc':>14}")
for name, fn in methods.items():
    t0 = time.perf_counter()
    Z = fn()
    print(f"{name:<8} {time.perf_counter()-t0:>8.1f} {knn_quality(Z, y):>14.3f}")
```

Output:

```
method    seconds  2-D 15-NN acc
PCA (2)       0.0          0.641
LDA (2)       0.0          0.687
t-SNE         6.5          0.976
UMAP         14.6          0.978
```

Listing 9 — Association rules from scratch: the lift lesson

Apriori's level-wise search using downward closure, then every rule scored. The teaching moment is in the table: beer → milk has confidence 0.80 but lift 1.00 — milk is simply everywhere, the rule is noise. beer → diapers, same confidence, lift 1.33 — a real association. Never read confidence without lift.

```
"""Listing 9: association rules from scratch -- support, confidence, lift on
baskets."""
from itertools import combinations

baskets = [
    {"bread", "milk"}, {"bread", "diapers", "beer", "eggs"},
    {"milk", "diapers", "beer", "cola"}, {"bread", "milk", "diapers", "beer"},
    {"bread", "milk", "diapers", "cola"}, {"bread", "milk"},
    {"milk", "diapers", "beer"}, {"bread", "cola"}, {"bread", "milk", "diapers"},
    {"milk", "beer"},
]
n = len(baskets)
def support(itemset):
    return sum(itemset <= b for b in baskets) / n

# Apriori level-wise search: an itemset can be frequent only if all subsets are
min_sup = 0.3
items = sorted({i for b in baskets for i in b})
L1 = [frozenset([i]) for i in items if support(frozenset([i])) >= min_sup]
L2 = [frozenset(c) for c in combinations(sorted({i for s in L1 for i in s}), 2)
      if support(frozenset(c)) >= min_sup]
print("frequent single items:", sorted(sorted(s)[0] for s in L1))
print("frequent pairs       :", [tuple(sorted(s)) for s in L2])

print(f"\n{'rule':<22} {'support':>8} {'confidence':>11} {'lift':>6}")
for pair in L2:
    for a in pair:
        A, B = frozenset([a]), pair - frozenset([a])
        sup, conf = support(pair), support(pair) / support(A)
        lift = conf / support(B)
        b = list(B)[0]
        print(f"{a:>8} -> {b:<10} {sup:>8.2f} {conf:>11.2f} {lift:>6.2f}")
```

Output:

```
frequent single items: ['beer', 'bread', 'cola', 'diapers', 'milk']
frequent pairs       : [('beer', 'diapers'), ('beer', 'milk'), ('bread', 'diapers'),
('bread', 'milk'), ('diapers', 'milk')]

rule                    support  confidence   lift
    beer -> diapers        0.40        0.80   1.33
 diapers -> beer           0.40        0.67   1.33
    beer -> milk           0.40        0.80   1.00
    milk -> beer           0.40        0.50   1.00
   bread -> diapers        0.40        0.57   0.95
 diapers -> bread          0.40        0.67   0.95
   bread -> milk           0.50        0.71   0.89
    milk -> bread          0.50        0.62   0.89
    milk -> diapers        0.50        0.62   1.04
 diapers -> milk           0.50        0.83   1.04
```

Listing 10 — Anomaly detection: three detectors, scored against planted truth

An injection study: 950 normal points, 50 planted outliers, no labels used in fitting. All three detectors take a contamination/ν knob — the chosen alarm rate — and precision/recall land accordingly. LOF and Isolation Forest lead here; One-Class SVM trails, true to its reputation as the fussiest.

```
"""Listing 10: anomaly detection -- Isolation Forest vs One-Class SVM vs LOF,
scored."""
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import precision_score, recall_score

rng = np.random.default_rng(9)
X_norm, _ = make_blobs(n_samples=950, centers=3, cluster_std=1.0, random_state=9)
X_anom = rng.uniform(X_norm.min(0) - 4, X_norm.max(0) + 4, (50, 2))   # scattered
outliers
X = StandardScaler().fit_transform(np.vstack([X_norm, X_anom]))
y = np.hstack([np.zeros(950), np.ones(50)])                            # 1 = anomaly

detectors = {
    "IsolationForest": IsolationForest(contamination=0.05, random_state=9),
    "One-Class SVM": OneClassSVM(nu=0.05, gamma="scale"),
    "LOF": LocalOutlierFactor(n_neighbors=25, contamination=0.05),
}
print(f"{'detector':<16} {'precision':>10} {'recall':>7}")
for name, det in detectors.items():
    pred = det.fit_predict(X)              # -1 = anomaly in sklearn convention
    flag = (pred == -1).astype(int)
    print(f"{name:<16} {precision_score(y, flag):>10.2f} {recall_score(y,
flag):>7.2f}")
```

Output:

```
detector          precision  recall
IsolationForest        0.82    0.82
One-Class SVM          0.71    0.70
LOF                    0.84    0.84
```

Pitfalls, comparisons and practical tips

The clustering algorithms, side by side:

	k-means	Hierarchical	DBSCAN/ HDBSCAN	GMM
Cluster definition	near a centroid	merge-tree cut	density-connected region	Gaussian component
Needs k	yes	no (cut later)	no	yes (BIC helps)

	k-means	Hierarchical	DBSCAN/ HDBSCAN	GMM
Shapes	spherical/ convex	linkage-dependent	arbitrary	ellipsoidal
Noise/outliers	dragged means	outlier-sensitive (except single)	native noise label	soft, but singularities
Assignment	hard	hard	hard + noise	soft (responsibilities)
Scale (n)	excellent (mini-batch)	$O(n^2)$ — tens of thousands	good (index-accelerated)	moderate
Determinism	init lottery	deterministic	deterministic-ish	init lottery
Key risk	wrong-shape fiction	wrong linkage	ε cliff / density variation	singularities, local maxima

Classic traps:

- **Clustering unscaled data.** Every method here except the tree-free ones runs on distances; one wide-range feature owns the geometry silently (Chapter 6's lesson, third appearance — it keeps appearing because people keep doing it).
- **Trusting the elbow.** Real elbows are mush; inertia comparisons across k are structurally rigged toward more clusters. Silhouette, gap statistic, BIC (GMM), stability under resampling — and the business question "how many segments can we act on?" — before declaring k.
- **Validating with the objective you optimized.** "Inertia is low" praises k-means for being k-means. Use *external* checks: downstream task lift, label agreement where partial labels exist (ARI, as the listings do), human inspection of samples, stability across resamples.
- **Reading t-SNE geometry.** Island sizes and gaps are artifacts; perplexity changes the story; noise makes clusters. Show multiple perplexities, never measure distances on the plot, never feed the coordinates to models.
- **PCA without centering — or on unstandardized mixed units.** Uncentered SVD finds the mean direction, not variance; unstandardized PCA elects the largest-unit feature as PC1. Also: variance ≠ relevance — the label-discriminative direction can hide in late PCs (LDA exists for that).
- **DBSCAN with one ε on varying densities.** Structurally impossible fit; the k-distance plot picks ε for *one* density regime only. HDBSCAN is the modern default answer.
- **GMM singularities.** A component collapsing onto a point sends likelihood to ∞ — floor the covariances (reg_covar), sanity-check fitted σ's before believing a "great" likelihood.
- **Confidence without lift.** Listing 9's beer→milk: confidence 0.80, lift 1.00, information zero. Popular consequents make every rule look confident.

- **Anomaly contamination as gospel.** contamination=0.05 *defines* a 5% alarm rate; it doesn't discover the true anomaly fraction. Set it from triage capacity and cost asymmetry, and validate by injection or labeled samples.
- **Forgetting the pipeline order.** Reduce-then-cluster (PCA→k-means, UMAP→HDBSCAN) changes results and their meaning; the reduction is part of the model, so tune and report it as such — and fit it on training data only when any downstream supervised step exists (Chapter 4's leakage rule applies to *unsupervised* preprocessing too).

Default reaches. Segmenting compact numeric data: k-means++ with silhouette-guided k. Unknown shapes, noise expected: HDBSCAN. Need soft memberships, densities, or generative scoring: GMM with BIC over k and covariance types. Need a taxonomy: agglomerative + dendrogram. Visualization: UMAP (and a PCA beside it for the linear-truth check). Anomalies at scale: Isolation Forest first.

Interview questions and answers

Q1. Walk through Lloyd's algorithm and prove it converges. To what does it converge?

Alternate: (1) assign each point to its nearest centroid; (2) move each centroid to its cluster's mean. Convergence: both steps weakly decrease inertia — assignment is optimal given centroids by definition of "nearest"; the mean minimizes summed squared distance given assignments (differentiate $\Sigma\|x_i-\mu\|^2$, set to zero, get μ = mean). Inertia is bounded below and there are finitely many partitions, so the descent terminates. But only at a *local* minimum — which one depends on initialization (Listing 1: 1.24× inertia spread over 20 random inits). Hence restarts and k-means++. *Interviewers listen for: the two "each step is optimal given the other" arguments — that's the proof.*

Q2. What is k-means++ and why does it work?

An initialization scheme: pick the first centroid uniformly at random; pick each subsequent centroid from the data with probability proportional to $D(x)^2$ — squared distance to the nearest already-chosen centroid. Far-from-covered regions are likely to get the next seed, so seeds spread across the data's structure instead of clumping (random init's failure: two seeds in one true cluster, zero in another, and Lloyd's local descent can't recover). It carries a provable guarantee — expected inertia within O(log k) of optimal before Lloyd's even runs — and costs one pass per seed. sklearn's default for good reason.

Q3. Your stakeholder asks "how many customer segments are there?" — describe your actual workflow.

Triangulate, don't oracle: scale features; run k over a range with silhouette (peak) and inertia (elbow, held loosely — Listing 2 shows why silhouette is sturdier); check BIC with a GMM as a likelihood-honest second opinion; test stability (recluster on bootstrap resamples — real segments recur, artifacts don't); and crucially invert the question — segments are for *acting on*, so 4 actionable segments beat 11 statistically-pretty ones. Present 2–3 candidate resolutions with profiles, not one number. Saying "the data may be hierarchical — 2 super-segments containing 6 sub-segments, both true" signals maturity, since cluster count is scale-dependent.

Q4. Name k-means's implicit assumptions and what breaks when each fails.

Squared-Euclidean-to-centroid implies: spherical clusters (fails on stretched/anisotropic data — boundaries cut across the grain; Listing 3: ARI 0.430), similar sizes and densities (fails with one big diffuse + small tight clusters — splitting the big one buys more inertia; ARI 0.252), convexity (fails on moons/rings — no centroid represents a crescent; ARI 0.249), and comparable feature scales. Fixes map one-to-one: GMM with full covariance for anisotropy and unequal spread, DBSCAN/spectral for non-convexity, standardization for scale. The strong-answer close: k-means always returns k clusters, whether or not they exist — it reports its assumptions, not your data.

Q5. Define the silhouette score and its failure modes.

Per point: a = mean intra-cluster distance, b = mean distance to the nearest other cluster; $s = (b-a)/\max(a,b) \in [-1,1]$; report the mean (and per-cluster means — one bad cluster hides in a global average). Near 1: tight and separated; near 0: on a boundary; negative: likely misassigned. Failures: presumes compact geometry, so density-chain clusters (moons) score poorly even when perfect; prefers coarse resolutions when groups overlap (can peak at 2 super-clusters); $O(n^2)$ cost bites at scale (subsample); and comparing silhouettes across different distance metrics or preprocessing is meaningless. Use it to compare k under one fixed pipeline, not as an absolute quality certificate.

Q6. Agglomerative linkages — describe each and give a case where single linkage is the best choice and one where it's the worst.

Single: cluster distance = closest pair (chains, filament-friendly, noise-bridge-vulnerable). Complete: farthest pair (compact, outlier-sensitive). Average: mean pairwise. Ward: merge minimizing the increase in within-cluster variance (k-means's sibling; the default). Listing 4's dissociation is the answer template: on two moons single linkage is *perfect* (1.000 — crescents are chains) while ward is worst (0.162); on compact blobs with scattered noise, single chains through the gaps (0.571) while ward/average are near-perfect (0.993). Linkage = your prior about cluster shape; pick it from the geometry you expect.

Q7. How do you read a dendrogram, and how do you choose where to cut it?

Leaves are points; each merge is a horizontal join at a height equal to the inter-cluster distance at merge time; cutting at any height yields the clusters below the cut. Choose the cut at a large gap in merge heights — many cheap merges then a sudden expensive one means "these groups resisted merging" (Listing 4: distances 13.0 → 85.1; cut in the gap for k=3, recovering exact true sizes). Also legitimate: cut for a fixed k the business needs, or by minimum cluster size. Mention cophenetic correlation as the "does the tree respect the original distances?" check if asked how to validate the hierarchy itself.

Q8. Explain DBSCAN's core/border/noise taxonomy and its two parameters.

Core point: ≥ min_samples points within radius ε (density above threshold). Border: within ε of a core point but not core itself — joins the cluster, extends nothing. Noise: neither — labeled −1, a first-class output. Clusters are maximal sets connected through core points (density-reachability chains). min_samples sets the density bar (rule of thumb ≥ d+1, larger for noisy data); ε sets the scale and is the sensitive one — Listing 5's sweep goes shatter (34 fragments) → perfect → single blob across 0.05→0.4. Pick ε from the knee of the sorted k-distance plot; or skip the problem with HDBSCAN.

Q9. What problem does HDBSCAN solve that DBSCAN structurally cannot?

Clusters of different densities under one global ε: an ε tight enough for the dense cluster labels the sparse cluster noise; loose enough for the sparse one merges the dense pair. HDBSCAN examines all density levels at once — conceptually the full hierarchy of DBSCAN solutions over ε — and extracts clusters that persist across the widest range of densities (stability-based selection). Result: no ε; min_cluster_size is the main knob and it's interpretable ("smallest group I care about"). Listing 5: tuned-DBSCAN quality (ARI 0.967) with nothing tuned. Bonus: it yields per-point membership strengths and an outlier score, and pairs famously with UMAP embeddings.

Q10. Derive the E and M steps for a GMM and state EM's convergence guarantee precisely.

E-step: responsibilities $r_{ij} = \pi_j N(x_i|\mu_j,\Sigma_j) / \Sigma_l \pi_l N(x_i|\mu_l,\Sigma_l)$ — Bayes' rule for "which component made this point", with parameters frozen. M-step: with $N_j = \Sigma_i r_{ij}$ — $\pi_j = N_j/n$, $\mu_j = \Sigma_i r_{ij}x_i/N_j$, $\Sigma_j = \Sigma_i r_{ij}(x_i-\mu_j)(x_i-\mu_j)^T/N_j$ — Gaussian MLE with fractional counts. Guarantee: each full iteration cannot decrease the observed-data log-likelihood (the E-step builds a lower bound tight at current parameters via Jensen's inequality; the M-step maximizes it), so EM converges monotonically — to a local maximum or saddle, not necessarily global (Listing 6 asserts the monotonicity in code and watches −7746 → −2161). Follow-ups to expect: singularities (floor Σ), init from k-means, restarts.

Q11. In what precise sense is k-means a special case of GMM?

Constrain the GMM: equal weights, shared spherical covariance εI, and take ε → 0. The responsibilities $r_{ij} \propto \exp(-\|x_i-\mu_j\|^2/2\varepsilon)$ then concentrate all mass on the nearest centroid — soft assignments harden into Lloyd's assignment step — and the M-step's weighted mean becomes the plain cluster mean, Lloyd's update step. So k-means is EM on a degenerate GMM with hard assignments and no covariance learning. The practical reading: choosing k-means over GMM is asserting spherical, equal-variance clusters and refusing uncertainty — sometimes exactly right (speed, simplicity), but it should be a choice, not a default (Listing 3 shows the price when it's wrong).

Q12. Why can't you fit a GMM by just differentiating the log-likelihood and using gradient descent?

You can, actually — the objection is subtler and stating it well scores points. The log-likelihood $\log \Sigma_j \pi_j N(x|\theta_j)$ is differentiable; but the sum inside the log couples all components (no per-component decomposition), constraints must be maintained (π on the simplex, Σ positive-definite — requiring reparameterization), and the surface has the same local maxima *plus* the singularity problem in raw form. EM handles all of this structurally: the E-step decouples the components (given responsibilities, each component's M-step is an independent, closed-form, constraint-respecting weighted MLE), no learning rate exists, and monotone improvement is guaranteed. Gradient/second-order methods do get used (and can converge faster near optima); EM wins on robustness, simplicity, and closed-form steps.

Q13. Derive PCA as a variance-maximization problem. Why do eigenvectors appear?

Seek unit vector w maximizing the variance of the projection: $\mathrm{Var}(Xw) = w^T\Sigma w$ with Σ the covariance matrix. Lagrangian for the constraint $\|w\|=1$: $w^T\Sigma w - \lambda(w^Tw - 1)$; stationarity gives $\Sigma w = \lambda w$ — an eigenvector equation. The variance achieved is $w^T\Sigma w = \lambda$, so the maximizer is the eigenvector with the largest eigenvalue; subsequent components repeat the argument orthogonal to those already chosen. Equivalent view: minimizing squared reconstruction error yields the same subspace (Pythagoras splits total variance into kept + lost). In practice compute by SVD of centered data — never form Σ explicitly for conditioning reasons (Listing 7 matches sklearn exactly this way).

Q14. Why must data be centered before PCA, and when should it also be standardized?

PCA analyzes the covariance structure — spread around the mean. Uncentered, the "first component" of X points at the data's mean (the dominant direction of raw second moments), not along its variance; results are geometrically meaningless. Standardize (unit variance per feature) when features have different units or scales: variance is unit-dependent, and a feature in centimeters has 10^4 times the variance of the same feature in meters — raw-scale PCA elects loud-unit features as PCs. Equivalent statement: standardized PCA = eigendecomposition of the correlation matrix. Keep raw scales only when units are shared and magnitude differences are genuinely meaningful (e.g. spectra, pixel intensities).

Q15. PCA vs LDA — objectives, constraints, when each wins.

PCA: unsupervised, finds directions of maximal total variance; up to d components; ignores labels entirely. LDA: supervised, finds directions maximizing between-class scatter over within-class scatter; at most (classes − 1) components (the between-class scatter matrix's rank). When labels exist and the goal is class structure, LDA's axes are chosen for exactly that (Listing 8: 0.687 vs 0.641 in 2-D on digits); PCA can bury the discriminative direction in a late component since variance ≠ relevance. PCA wins when there are no labels, when you need more components than classes allow, as generic decorrelation/compression, or when LDA's Gaussian-equal-covariance assumptions are badly violated. Common pipeline: PCA to denoise, then LDA.

Q16. Your colleague measures the distance between two clusters on a t-SNE plot to argue they're "more different" than another pair. Respond.

The measurement is invalid. t-SNE preserves local neighborhoods and deliberately sacrifices global geometry: inter-cluster distances and cluster sizes in the embedding are artifacts of the optimization (the heavy-tailed low-D kernel and per-point bandwidths equalize densities and repel non-neighbors indiscriminately). The same data at different perplexities rearranges the islands. What the plot does support: points in the same island are mutual neighbors in high-D. For the colleague's actual question, compute distances in the original (or PCA) space, or use a method with better global fidelity (UMAP is better but not exempt) and confirm quantitatively. Listing 8's framing helps: t-SNE's 0.976 is *neighborhood* preservation, nothing more.

Q17. Compare t-SNE and UMAP for a practitioner.

Both build neighbor graphs in high-D and lay them out in low-D; differences that matter: UMAP is typically faster at scale, preserves global structure somewhat better, and — decisive for production — has a transform() for new points, while classic t-SNE must re-fit (out-of-sample extensions exist but aren't standard). t-SNE's perplexity ↔ UMAP's n_neighbors (local-global dial); UMAP adds min_dist (visual packing). Both: stochastic (seed-dependent layouts), axes meaningless, densities distorted, "clusters" appear in noise. Listing 8: equal neighborhood quality (0.976 vs 0.978) on digits. Default: UMAP for routine work and pipelines; t-SNE remains standard in some fields (single-cell bio) and for small careful visualizations.

Q18. Define support, confidence, and lift. Why is high confidence alone misleading?

For rule A→B over baskets: support = P(A,B) — pattern frequency; confidence = P(B|A) — implication reliability; lift = P(A,B)/(P(A)P(B)) — co-occurrence relative to independence. Confidence inherits the consequent's base rate: if B is in 80% of baskets, almost *any* A gives confidence ≈ 0.8 (Listing 9: beer→milk, confidence 0.80, lift 1.00 — pure base rate; beer→diapers, same confidence, lift 1.33 — real signal). Lift corrects by dividing out P(B): >1 complements, 1 independent, <1 substitutes. Caveats worth adding: lift is symmetric (loses direction), unstable at tiny supports (screen by support first), and none of the three implies causation.

Q19. Explain the Apriori property and how FP-Growth improves on Apriori.

Apriori (downward closure): every subset of a frequent itemset is frequent — equivalently, adding items never raises support. This prunes the exponential itemset lattice: candidate k-itemsets need only be built from frequent (k−1)-itemsets, and any candidate with an infrequent subset is dead on arrival (Listing 9's level-wise construction). Apriori's costs: multiple database scans and huge candidate sets at low support. FP-Growth removes candidate generation: compress transactions into an FP-tree (prefix tree with counts; frequent items sorted so prefixes share), then recursively mine conditional subtrees — two scans total, typically much faster on large sparse data. Same output, different search.

Q20. How does Isolation Forest decide something is anomalous, and why is it the industrial default?

Inverted logic: instead of modeling "normal", it measures how easy a point is to *isolate* — grow trees with random feature/threshold splits; a point's anomaly score comes from its average path length to isolation across trees (outliers strand alone in few splits; inliers need many). Default-worthy properties: O(n log n) with small constants; no distance computations (no scaling sensitivity, tolerates high dimension and irrelevant features better than KNN-family methods); handles mixed magnitudes; trivially parallel; subsampled trees make it robust. Weaknesses: axis-aligned splits miss some structured anomalies; scores need calibration to costs (contamination = chosen alarm rate — Listing 10). Interview bonus: it's an ensemble (Chapter 7) whose members are deliberately *random*, not accurate — diversity without skill, because the signal is structural.

Q21. LOF vs Isolation Forest — when does the local method matter?

LOF scores each point by the ratio of its neighbors' local density to its own — anomalous means "sparse relative to *its own neighborhood*". That relativity is the point: a reading at the edge of a tight cluster can be globally unremarkable (denser than the sparse cluster's core) yet locally anomalous; global methods (Isolation Forest, GMM likelihood) miss it, LOF catches it. Costs: KNN machinery — $O(n^2)$-ish without indexes, scale-sensitive, curse-of-dimensionality-fragile, and the classic version doesn't score unseen points (novelty=True refits the contract). Practical rule: Isolation Forest first for scale and robustness; LOF when clusters of very different densities are known to exist and local context defines "anomalous" (Listing 10: 0.84/0.84 — best in that geometry).

Q22. How do autoencoders detect anomalies, and what silently breaks the approach?

Train a bottlenecked encoder-decoder to reconstruct (mostly normal) data; the bottleneck forces learning the normal manifold, so unseen anomalies reconstruct badly — reconstruction error is the anomaly score (details in Chapter 18). Silent breakers: anomalies contaminating training (the AE learns to reconstruct them too — clean or robustify the training set); over-capacity (a too-wide/deep AE reconstructs *everything*, including anomalies — the bottleneck must actually bottleneck); anomalies lying *on* the normal manifold (fraudulent transactions crafted to look typical — reconstruction can't see intent); and threshold drift as normal behavior evolves (monitor the error distribution, Chapter 27). Fair summary: powerful for high-dimensional structured data (images, sequences) where classical detectors' distances fail; overkill for 10 tabular features.

Q23. How do you evaluate a clustering when no labels exist — and when partial labels exist?

No labels: internal metrics (silhouette, Davies-Bouldin) — knowing they reward the geometry the algorithm already optimized; stability — recluster under resampling/perturbation, measure agreement (ARI between runs), since real structure recurs; downstream utility — does adding cluster-ID as a feature lift a supervised task, do segments move a business metric in A/B; and human triage of samples per cluster. Partial labels: external indices — ARI (chance-corrected pair agreement; the listings' metric), NMI (information-theoretic), purity — computed on the labeled subset. The interview frame: internal metrics are sanity checks, stability is evidence, downstream utility is proof; and any evaluation must hold the preprocessing/distance fixed or it compares pipelines, not clusterings.

Q24. Why does high dimensionality hurt clustering, and what are the standard mitigations?

Distance concentration (Chapter 4): as d grows with independent-ish features, all pairwise distances converge to the same value, so "nearest" and "densest" lose contrast — k-means boundaries become arbitrary, DBSCAN finds one blob or all noise (density thresholds are distance-based), silhouettes flatten. Mitigations: reduce first — PCA to the elbow of explained variance, or UMAP for nonlinear structure (with the caveat that the reduction now shapes the answer); feature-select with domain knowledge; use cosine distance on sparse/directional data (text — Chapter 19); subspace/spectral clustering when clusters live in different low-D subspaces; and for mixtures, restrict covariances (diagonal) so parameters don't explode. The honest note: if informative structure is confined to a few dimensions, finding *those* is the clustering problem.

Q25. Design the unsupervised half of a fraud pipeline: you have 10M transactions, no labels yet, and analysts who can review 200 cases/day.

Scope by the review budget: 200/day over 10M/period fixes the alarm rate — that's the contamination knob, chosen not discovered (Listing 10's lesson). Detector: Isolation Forest on engineered features (amounts, velocities, novelty counters — Chapter 9) for scale; add a GMM/density score per customer segment (cluster first — behavior is heterogeneous, and "anomalous for a student account" ≠ "for a business account": LOF's local logic at the segment level). Rank, don't threshold: send the top-200 to analysts daily. Their verdicts become labels — the pipeline's real product — feeding a supervised model (Chapter 7's boosted trees) that gradually takes over; keep the unsupervised layer for novel-pattern detection forever. Monitor score drift (Chapter 27). Every design choice here traces to a chapter concept, which is why the question gets asked.

Q26. A GMM fit reports spectacular log-likelihood, and one component has variance 1e-9. Diagnose.

A singularity, not a discovery: a component has collapsed onto one point (or a few duplicates); as σ→0 that point's density →∞, so the likelihood diverges — the "spectacular" number is the pathology itself. The MLE for mixtures is technically unbounded; EM found the degenerate direction. Fixes: floor the covariance (reg_covar), use MAP-EM with a prior (inverse-Wishart shrinkage — Bayesian regularization, Chapter 1's MAP again), remove duplicate points, reduce k, restart from different inits and prefer solutions with sane σ's. Then re-select k by BIC computed only over non-degenerate fits. Recognizing "likelihood too good = broken" is the point of the question.

Q27. Implement one k-means assignment step in two lines of NumPy, given X (n×d) and centers C (k×d).

`d2 = ((X[:, None, :] - C[None, :, :]) ** 2).sum(-1)` — broadcasting to an (n, k) matrix of squared distances — then `labels = d2.argmin(1)`. Volunteer the production notes: the broadcast materializes n×k×d — for large n use the expansion $\|x\|^2 - 2XC^T + \|c\|^2$ to stay at n×k (`d2 = (X**2).sum(1)[:,None] - 2*X@C.T + (C**2).sum(1)`); skip the square root (argmin is monotone-invariant); and this exact computation is the inner loop of vector quantization and IVF index training in vector databases (Chapter 24).

Q28. When would you deliberately choose plain k-means over every fancier option in this chapter?

When its assumptions plausibly hold (roundish, similar-scale groups in scaled numeric features) or when its *role* doesn't require them to: as a quantizer (codebooks, IVF cells, minibatch-friendly embedding compression — correctness is "low distortion", not "true clusters"); at scales where $O(n^2)$ methods and full-covariance GMMs are unaffordable; when determinism-enough (k-means++ + n_init) and five-line explainability matter for handoff; as the init for GMM/EM; or as the honest baseline every fancier clustering must beat (Chapter 7's baseline discipline, unsupervised edition). The disciplined phrasing: k-means is a fast biased estimator of cluster structure — ideal when speed matters and the bias is known to be tolerable.